# Linear Algebra and its Applications
## Assignment 3
Due: 19 February 2026

---

Write a your own $\mathsf{LU}$ solver in Python, based on Gaussian elimination with partial pivoting. The input is a square matrix $A$ and a right hand side vector $b$ (both randomly generated). The output should be $L, U$, the (row) permutation matrix and the solution to the problem $Ax = b$. Once you are sure that the code is working properly you should experiment with various sizes and note the time taken. Prepare comparison tables (one for factorization and the other for the solution, i.e., forward and backward substitution) that shows the time needed for your algorithm and also for the SciPy's $\mathsf{LU}$ solver. For each system you should also note down how far were you from the actual solution. [30 points]

To be precise, here are some guidelines.

- There is a built-in $\mathsf{LU}$ solver in SciPy. However, you should write your own solver without calling any readily available matrix factorization routine from any of the Python libraries.
- In particular, your solver and factorization should be implemented as different functions. The $\mathsf{LU}$ factorization should use partial pivoting. If no nonzero pivots exist, stop and report that the matrix is singular. The solver function should use forward and backward substitution.
- Use the NumPy random library to generate an $n \times n$ real matrix for various values of $n$, like $n = 100, 500, 1000, 5000, \ldots$ etc. (take at least 5 different values, more the better)
- The ideal output of the program should be the permutation matrix $P$, the unit lower triangular $L$ and the upper triangular $U$. Of course, you don't want to print these matrices individually.
- Measure the time taken just for the factorization (don't count matrix initialization, verification etc.) and separately for $Ax = b$ solver. Time both the functions, yours as well as the SciPy's.
- Perform accuracy and stability analysis by computing the following quantities for both the solvers. Present these findings as a table comparing both the routines:
  1. Backward error of factorization $\dfrac{||PA - LU||}{||A||}$.
  2. Residual $\frac{||A\hat{x} - b||}{||b||}$.

Implementation guidelines

- You should write an `helper_functions.py`" which will have all the auxiliary functions for your implementation.
- Document each of your functions well (at least one line for each).
- Write the "`results.ipynb`" file, which will import the "`helper_functions.py`" file and display the results.
- use an understandable naming convention for your functions (don't use one alphabet for defining functions and variables).

- The pdf of the results notebook should be named "your_name_roll_no.pdf"

**Note**: Your submission should be a single PDF containing your Juypter Notebook, python files and comparison tables. Make sure that unnecessary things like, error messages, trial runs etc. are not included in the submission.